

Computer Networks - Complete Notes

Coal India Limited - Management Trainee (Systems) | CBT Preparation

Header watermark: Asabhya Maanav

Table of Contents

Network Fundamentals	8
Definition, Goals and Applications	8
Network Types: LAN, MAN, WAN, PAN	8
Topologies	8
Transmission Modes	9
Connection-Oriented vs Connectionless	9
Performance Metrics	9
Layered Architecture	11
Need for Layering; Service vs Protocol vs Interface	11
OSI Model (7 Layers)	11
TCP/IP Model	11
OSI vs TCP/IP	11
Encapsulation, Decapsulation and PDUs	12
Physical Layer Basics	13
Analog vs Digital Signals	13
Transmission Media	13
Nyquist and Shannon Capacity	13
Bit Rate vs Baud Rate	13
Encoding Schemes	14
Multiplexing	14
Switching	15
Circuit Switching	15
Packet Switching	15
Message Switching	15
Comparison	15
Data Link Layer Services	16
Core Services	16
LLC vs MAC Sublayer	16
Framing	16
Framing Methods	16
Error Detection and Correction	16
Error Types	16

Hamming Distance	16
Parity Check	17
Checksum (Internet Checksum)	17
Cyclic Redundancy Check (CRC)	17
CRC Detection Capability	17
Hamming Code (Single-bit Correction)	18
Flow and Error Control Protocols	19
Stop-and-Wait	19
Stop-and-Wait ARQ	19
Sliding Window: Go-Back-N ARQ	19
Sliding Window: Selective Repeat ARQ	19
Window Size vs Sequence-Number Bits	19
Piggybacking	19
Efficiency / Throughput Computations	20
Optimal Window Size and BDP	20
Medium Access Control (MAC)	20
Static vs Dynamic Allocation	20
Random Access: ALOHA	20
Random Access: CSMA	20
CSMA/CD	20
CSMA/CA	21
Controlled Access	21
Channelization	21
Ethernet and LAN Devices	22
IEEE 802.3 Ethernet Frame Format	22
Min/Max Frame Size	22
MAC Addressing	22
Fast / Gigabit Ethernet	22
Devices and Layers	22
Collision vs Broadcast Domain	22
Spanning Tree Protocol	22
VLAN and 802.11 WLAN	22
Network Layer Services	24
Core Functions	24
Forwarding vs Routing	24

Datagram vs Virtual Circuit (Network Layer)	24
IPv4 Addressing	24
Address Structure	24
Classful Addressing	24
Network/Host IDs and Special Addresses	24
Private IP Ranges	24
Classless Addressing (CIDR)	25
Subnetting	25
VLSM	25
Supernetting / Route Aggregation	25
Address Block Problems	25
IPv4 Datagram	26
Header Fields	26
TTL, Protocol, Checksum, Total Length	26
Fragmentation and Reassembly	26
IPv6	26
128-bit Addressing	26
Header Simplification	26
Transition	26
Routing Algorithms	28
Static vs Dynamic Routing; Metrics	28
Shortest Path Routing (Dijkstra)	28
Flooding	28
Distance Vector Routing	28
Link State Routing	28
Hierarchical Routing, AS, BGP	28
IP Support Protocols	29
ARP (Address Resolution Protocol)	29
RARP	29
ICMP	29
DHCP	29
NAT	29
Transport Layer Services	30
Process-to-Process Delivery; Ports and Sockets	30

Multiplexing and Demultiplexing	30
Connection-oriented vs Connectionless	30
UDP	30
UDP Header	30
UDP Checksum	30
Use Cases	30
TCP	30
TCP Header Fields	30
Sequence and Acknowledgment Numbers	30
Connection Management	31
Flow Control	31
Error Control	31
Congestion Control	31
cwnd Evolution Computation	31
TCP vs UDP	31
Sockets	32
Socket = IP + Port	32
Port Ranges	32
DNS (Domain Name System)	33
Namespace Hierarchy	33
Record Types	33
Iterative vs Recursive Resolution	33
DNS Caching, TTL, Port	33
Email Protocols	33
Email Architecture	33
SMTP	33
POP3	33
IMAP	34
MIME	34
Web (HTTP/HTTPS)	34
Request/Response Model	34
Methods	34
Status Codes	34
Persistent vs Non-persistent	34

Stateless; Cookies/Sessions	34
HTTPS/TLS	34
File Transfer (FTP)	35
Control vs Data Connection	35
Active vs Passive Mode	35
ASCII vs Binary Transfer	35
Other Services	35
Telnet / SSH	35
SNMP	35
Proxy / Web Caching	35
Network Security: Foundations	37
Security Goals (CIA + AN)	37
Active vs Passive Attacks	37
Cryptography	37
Symmetric-Key Cryptography	37
DES (Data Encryption Standard)	37
AES (Advanced Encryption Standard)	37
Asymmetric-Key (Public-Key) Cryptography	38
RSA Algorithm	38
Diffie-Hellman Key Exchange	38
Hashing and Integrity	38
Cryptographic Hash Functions	38
MAC and HMAC	39
Digital Signatures and Certificates	39
Digital Signatures	39
Digital Certificates and PKI	39
SSL/TLS	39
SSL/TLS Overview	39
Firewalls and IDS	39
Firewalls	40
IDS / IPS	40
Common Attacks	40

Attack Catalog	40
CIL MT Exam Focus / Practice Checklist	42
Port Numbers and Protocol-to-Layer Mapping	42
Formula Quick Sheet	42
Subnetting / CIDR Computations	42
Fragmentation Offset/Size Computations	42
Sliding Window Efficiency	43
TCP Congestion Window Evolution	43
CRC and Hamming Computation	43
Delay / Throughput / BDP	43
Header Field Identification	43

Network Fundamentals

Definition, Goals and Applications

- A **computer network** is a collection of **autonomous** computing devices interconnected by a single technology so they can **exchange data** and **share resources**.
- "Autonomous" means **no master-slave** relationship; if one machine can forcibly start/stop another it is not a network but a **distributed system** variant.
- Primary goals: **resource sharing**, **high reliability** (replication/alternate routes), **scalability**, **cost saving**, **communication medium**, and **load distribution**.
- A **distributed system** is a **software** (application-layer) layer built on top of a network that hides the network from the user; a **network** is the **hardware/protocol** substrate. Distinction is the **level of abstraction**.
- Applications: **business/resource sharing**, **home (access to remote info, e-commerce, IPTV)**, **mobile (m-commerce, sensor nets)**, and **social**.

Memory Trick

- Goals = "RRSC": **Reliability**, **Resource sharing**, **Scalability**, **Cost**.

CBT Trap

- Network** vs **distributed system**: difference is **abstraction/software**, not hardware. The WWW runs on the Internet (**distributed system on a network**).

Network Types: LAN, MAN, WAN, PAN

- PAN (Personal Area Network)**: range about **1-10 m**; example **Bluetooth**, USB; smallest scope.
- LAN (Local Area Network)**: a **building/campus**, range up to **~1 km**; privately owned; **high speed (10 Mbps-10 Gbps)**, **low delay**, **low error**; tech **Ethernet/Wi-Fi**.
- MAN (Metropolitan Area Network)**: spans a **city**, up to **~10-100 km**; example **cable TV**, **DOCSIS**, **WiMAX**.
- WAN (Wide Area Network)**: **country/continent** scale; uses **public carriers**; lower speed, higher delay; the **Internet** is a WAN of WANs.
- Increasing order of geographic span: **PAN < LAN < MAN < WAN**.

Type	Span	Ownership	Example
PAN	1-10 m	Personal	Bluetooth
LAN	up to 1 km	Private	Ethernet
MAN	10-100 km	Private/Public	Cable/WiMAX
WAN	Country+	Public	Internet

CBT Trap

- LAN** typically uses **broadcast** channels; **WAN** uses **point-to-point (switched)** links. **Internetwork** = networks joined by **routers/gateways**.

Topologies

- Topology** = the physical/logical **arrangement** of nodes and links.
- Bus**: single backbone cable; needs **terminators**; cheap but a **single break** kills the segment; uses **CSMA/CD**; n nodes need **1** cable and **1** I/O port each.
- Star**: all nodes connect to a **central hub/switch**; needs n cables and n ports on hub; failure of a **link** affects only one node, but **hub failure** kills all; most common in modern LANs.

- **Ring**: each node connects to **2** neighbors forming a loop; uses **token passing**; **unidirectional** (or dual ring for fault tolerance); a single break can break the ring.
- **Mesh**: dedicated point-to-point link between every pair; needs **$n(n-1)/2$** duplex links and **(n-1)** ports per node; **most robust**, most expensive.
- **Tree**: hierarchical extension of star (star of stars); used in **cable TV**.
- **Hybrid**: combination, e.g., **star-bus**.

Topology	Cables	Ports/node	Reliability
Mesh	$n(n-1)/2$	n-1	Highest
Star	n	1	Hub is SPOF
Bus	1	1	Low (backbone SPOF)
Ring	n	2	Break-sensitive

Memory Trick

- Mesh links = **$nC2 = n(n-1)/2$** . For **n=5**: **10** links.

CBT Trap

- Number of **cables** in mesh = **$n(n-1)/2$** ; number of **ports** per device = **n-1**. Examiners swap these.

Transmission Modes

- **Simplex**: **one direction** only (e.g., keyboard to CPU, monitor); full bandwidth one way.
- **Half-duplex**: both directions but **one at a time** (e.g., **walkie-talkie**, CSMA/CD Ethernet hub).
- **Full-duplex**: both directions **simultaneously** (e.g., **telephone**, switched Ethernet); needs two channels or shared with separation.

CBT Trap

- **Half-duplex** shares one channel by turns; **full-duplex** needs **two** links or **frequency separation**. Capacity of full-duplex link is shared in both directions.

Connection-Oriented vs Connectionless

- **Connection-oriented**: **setup -> transfer -> teardown**; ordered, reliable; analogy **telephone**; example **TCP**, **virtual circuit**.
- **Connectionless**: each packet (**datagram**) routed **independently**; no setup; analogy **postal**; example **UDP**, **IP**.

Feature	Connection-oriented	Connectionless
Setup	Yes	No
Ordering	Guaranteed	Not guaranteed
Reliability	High	Best effort
Example	TCP/VC	UDP/IP

Performance Metrics

- **Bandwidth (bit rate)**: number of **bits per second** a link can carry; in Hz it is the frequency range. Digital bandwidth measured in **bps**.
- **Transmission delay (Tt)** = **L / B** = packet size (bits) / bandwidth (bps); time to **push all bits** onto the wire.
- **Propagation delay (Tp)** = **distance / propagation speed** = **d / v** ; v approx **2×10^8 m/s** in copper/fiber (**3×10^8** in vacuum).
- **Queuing delay**: time a packet waits in **router buffers**; variable, load-dependent.

- **Processing delay**: time to **examine header** and decide forwarding.
- **End-to-end delay** = $n(T_t + T_p + T_q + T_{proc})$ for n hops (store-and-forward).
- **Throughput**: **actual** rate of successful delivery; always $\leq$ **bandwidth**.
- **Bandwidth-Delay Product (BDP)** = **Bandwidth x RTT** (or $\times T_p$) = max **bits in flight**; sets **optimal window size**.
- **RTT (Round-Trip Time)** = $2 \times T_p$ (+ processing) = time for a bit to go and the ACK to return.

Delay	Formula	Depends on
Transmission	L/B	Packet size, bandwidth
Propagation	d/v	Distance, medium
Queuing	variable	Congestion
Processing	variable	Router speed

Memory Trick

- **T_t** depends on **size**; **T_p** depends on **distance**. BDP = "**pipe volume**" = bandwidth $\times$ delay.

CBT Trap

- **Transmission delay** uses **bandwidth** (bps); **propagation delay** uses **speed** (m/s). Doubling bandwidth halves **T_t**, not **T_p**. For long-fat networks **T_p dominates**.

Layered Architecture

Need for Layering; Service vs Protocol vs Interface

- Layering provides **modularity**, **abstraction**, easier **design/maintenance**, and **interoperability**.
- A **service** = set of operations a layer offers to the layer **above** it (the "what").
- A **protocol** = set of **rules/format** governing communication between **peer** entities on different machines (the "how").
- An **interface** = tells the upper layer how to **access** the service (primitives between adjacent layers on the **same** machine).
- **Peer-to-peer** communication is **virtual**; actual data flows **down** the sender stack and **up** the receiver stack.

CBT Trap

- **Service** = vertical (between layers); **protocol** = horizontal (between peers). A layer can change its protocol without changing the **service**.

OSI Model (7 Layers)

- **OSI = Open Systems Interconnection**, by **ISO (1984)**; **7** layers; a **reference** model (rarely implemented as-is).
- Bottom to top: **Physical, Data Link, Network, Transport, Session, Presentation, Application**.
- **Layer 1 Physical**: transmits **raw bits**; deals with **voltage, pins, data rate, media, topology**; PDU = **bit**.
- **Layer 2 Data Link**: **node-to-node** delivery; **framing, MAC, flow & error control**; PDU = **frame**; addresses = **MAC**.
- **Layer 3 Network**: **host-to-host** across networks; **logical addressing (IP), routing**; PDU = **packet/datagram**.
- **Layer 4 Transport**: **process-to-process**; **segmentation, flow/error control, multiplexing**; PDU = **segment (TCP)/datagram (UDP)**.
- **Layer 5 Session**: **dialog control, synchronization, checkpoints**.
- **Layer 6 Presentation**: **translation, encryption, compression** (syntax/semantics).
- **Layer 7 Application**: **user services** (HTTP, FTP, SMTP, DNS).

Memory Trick

- Bottom-up: "**Please Do Not Throw Sausage Pizza Away**" = Physical, Data link, Network, Transport, Session, Presentation, Application.
- Top-down: "**All People Seem To Need Data Processing**".

CBT Trap

- **Encryption/compression** = **Presentation**; **dialog/sync** = **Session**; **routing** = **Network**; **flow control** appears at both **Data Link** (node) and **Transport** (end-to-end).

TCP/IP Model

- **TCP/IP** = **4** layers (**DoD** model): **Application, Transport, Internet, Network Access (Link)**; or **5** layers if Physical separated.
- **Application** = OSI **5+6+7**; **Transport** = OSI **4** (**TCP/UDP**); **Internet** = OSI **3** (**IP**); **Link** = OSI **1+2**.
- The **Internet** layer's main protocol is **IP** (**connectionless**); the model was **descriptive** (built from protocols), unlike OSI which was **prescriptive**.

OSI vs TCP/IP

Aspect	OSI	TCP/IP
Layers	7	4 (or 5)
Origin	ISO , prescriptive	DoD , descriptive
Transport	Both reliable	TCP reliable, UDP unreliable
Use	Reference	Practical/Internet
Coupling	Strict layers	Loosely coupled

CBT Trap

- OSI has **7**, TCP/IP has **4**. OSI defines **services/interfaces/protocols** distinctly; TCP/IP does not. **Session** and **Presentation** have **no separate** layer in TCP/IP.

Encapsulation, Decapsulation and PDUs

- **Encapsulation**: each layer adds its **header** (and DLL adds a **trailer**) as data goes **down** the sender stack.
- **Decapsulation**: each layer **strips** its header going **up** the receiver stack.
- PDU names: Application = **data/message**, Transport = **segment/datagram**, Network = **packet/datagram**, Data Link = **frame**, Physical = **bits**.

Memory Trick

- PDU order down the stack: "**Data -> Segment -> Packet -> Frame -> Bits**".

CBT Trap

- Only **Data Link** adds a **trailer** (FCS); others add header only. **Segment** (TCP) vs **Datagram** (UDP/IP) naming is frequently tested.

Physical Layer Basics

Analog vs Digital Signals

- **Analog signal**: **continuous** values, infinitely many levels (e.g., voice).
- **Digital signal**: **discrete** levels (e.g., **0/1**); subject to **attenuation**, **noise**, **distortion**.
- **Bit interval** = time for one bit; **bit rate** = **1 / bit interval**.

Transmission Media

- **Guided (wired)**: **twisted pair**, **coaxial**, **fiber optic**.
- **Unguided (wireless)**: **radio**, **microwave**, **infrared**.
- **Twisted pair**: two insulated copper wires twisted to reduce **crosstalk**; **UTP** (Cat5e/Cat6) and **STP**; cheap; used in **LAN/telephone**; max **~100 m** for Ethernet.
- **Coaxial**: central conductor + shield; higher bandwidth than twisted pair; used in **cable TV**.
- **Fiber optic**: carries light; **single-mode** (long haul, laser) and **multi-mode** (short, LED); immune to **EMI**, huge bandwidth, low attenuation, hard to tap (**secure**); most expensive.
- **Wireless**: no medium; **omnidirectional** (radio) vs **unidirectional** (microwave); affected by **interference**.

Medium	Bandwidth	EMI immunity	Cost
Twisted pair	Low-Med	Low	Low
Coax	Medium	Medium	Medium
Fiber	Highest	Highest	High

CBT Trap

- **Single-mode fiber** = **laser**, long distance; **multi-mode** = **LED**, short distance. Fiber is immune to **EMI** and hardest to **tap**.

Nyquist and Shannon Capacity

- **Nyquist (noiseless)**: $C = 2 \times B \times \log_2(L)$ bps, where B = bandwidth (Hz), L = number of signal levels.
- **Shannon (noisy)**: $C = B \times \log_2(1 + S/N)$ bps; S/N is the **signal-to-noise** ratio (linear).
- **SNR(dB)** = $10 \times \log_{10}(S/N)$; convert before using Shannon.
- When both apply, take the **lower** (Shannon caps the realistic max; Nyquist tells how many levels are usable).

Memory Trick

- Nyquist = "**2B log₂ L**" (levels); Shannon = "**B log₂(1+SNR)**" (noise). Nyquist has **no noise**; Shannon has **no levels**.

CBT Trap

- For SNR given in **dB**, first compute $S/N = 10^{(dB/10)}$. With SNR very high, Shannon $\rightarrow 0.332 \times B \times \text{SNR(dB)}$ approx. Examiners mix up which formula uses **levels (L)**.

Bit Rate vs Baud Rate

- **Bit rate** = bits/second; **baud rate** = **signal units (symbols)/second**.
- **Bit rate** = **baud rate** $\times \log_2(L)$ = baud $\times$ bits-per-symbol.
- **Baud rate** $\leq$ **bit rate** always; they are equal only when **1 bit per symbol (L=2)**.

CBT Trap

- **Baud** $\leq$ bit rate. Bandwidth requirement is governed by **baud rate**, not bit rate.

Encoding Schemes

- **NRZ (Non-Return to Zero)**: voltage stays constant during bit; **NRZ-L** (level) and **NRZ-I** (invert on 1); problem: **no self-clocking**, baseline wander on long runs.
- **Manchester**: each bit has a **mid-bit transition**; **self-clocking**; used in **Ethernet 10 Mbps**; needs **2x bandwidth** (baud = 2 x bit rate).
- **Differential Manchester**: transition at **start** represents bit; always mid-bit transition for clock.
- **RZ (Return to Zero)**: signal returns to zero mid-bit; three levels.

CBT Trap

- **Manchester** is **self-clocking** but uses **double** the bandwidth. **NRZ** is bandwidth-efficient but loses sync on long **0/1** runs.

Multiplexing

- **Multiplexing**: combining multiple signals over one shared **medium**.
- **FDM (Frequency Division)**: **analog**; divides **bandwidth** into frequency bands; needs **guard bands**; used in radio/TV.
- **TDM (Time Division)**: **digital**; divides **time** into slots; **Synchronous TDM** (fixed slots, may waste) vs **Statistical/Asynchronous TDM** (dynamic, efficient).
- **WDM (Wavelength Division)**: FDM for **fiber** (light wavelengths); uses prism to combine/split.

Technique	Domain	Signal	Use
FDM	Frequency	Analog	Radio/TV
TDM	Time	Digital	T1/E1
WDM	Wavelength	Optical	Fiber

CBT Trap

- **FDM/WDM** share **frequency/wavelength**; **TDM** shares **time**. **Statistical TDM** allocates slots **on demand**, no wasted slots.

Switching

Circuit Switching

- **Circuit switching**: a **dedicated** physical path is reserved end-to-end for the whole session; phases = **setup** -> **data transfer** -> **teardown**.
- Resources are **reserved** (**guaranteed bandwidth**) even if idle (**wasteful**); example **PSTN telephone**.
- No **addressing** per packet during transfer; **constant delay**; no congestion after setup.

Packet Switching

- **Packet switching**: data split into **packets**; each carries **header/address**; statistical sharing of links.
- **Datagram** approach: each packet routed **independently**; may arrive **out of order**; **connectionless**; example **IP**.
- **Virtual circuit** approach: **setup** establishes a path; packets carry a **VC number/label**; ordered delivery; **PVC** (permanent) and **SVC** (switched/on-demand); example **Frame Relay, ATM, MPLS**.

Message Switching

- **Message switching**: entire **message** stored and forwarded hop by hop (**store-and-forward**); no size limit; high **delay**; obsolete (telegraph).

Comparison

Feature	Circuit	Datagram packet	Virtual circuit
Setup	Yes	No	Yes
Path	Dedicated	Per-packet	Fixed per VC
Order	Preserved	May reorder	Preserved
Address	At setup	Full in every packet	VC id
Efficiency	Low	High	High

Memory Trick

- Circuit = "**phone call**"; Datagram = "**postcards**"; VC = "**reserved train track**".

CBT Trap

- **Virtual circuit** still does **store-and-forward** but uses **small VC labels** instead of full addresses. **Datagram** puts **full destination address** in every packet. Circuit switching reserves bandwidth even when **idle**.

Data Link Layer Services

Core Services

- The **Data Link Layer (DLL)** provides **node-to-node (hop-to-hop)** delivery over a single link.
- Functions: **framing**, **physical (MAC) addressing**, **flow control**, **error control**, **access control (MAC)**.
- **Flow control** matches sender speed to receiver capacity; **error control** handles damaged/lost frames via **ARQ**.

LLC vs MAC Sublayer

- **IEEE 802** splits DLL into **LLC (Logical Link Control, 802.2)** on top and **MAC (Media Access Control)** below.
- **LLC**: provides **flow/error control** and interface to network layer; **MAC**: controls **who accesses the medium** and does **framing/addressing**.

CBT Trap

- **LLC** is the **upper** sublayer (interface to L3); **MAC** is the **lower** sublayer (access to medium). MAC addresses live in the **MAC** sublayer.

Framing

Framing Methods

- **Framing**: dividing the bit stream into distinguishable **frames**.
- **Fixed-size framing**: no delimiters needed (e.g., **ATM cells = 53 bytes**).
- **Variable-size framing**: needs to mark frame **boundaries**.
- **Character/byte count**: first field gives frame **length**; fragile - a corrupted count **desynchronizes** everything.
- **Byte/character stuffing**: a **FLAG** byte marks boundaries; an **ESC** byte is stuffed before any data byte matching FLAG/ESC; byte-oriented (PPP).
- **Bit stuffing**: flag = **01111110** (0x7E); sender inserts a **0** after **five consecutive 1s** in data; receiver removes it; bit-oriented (**HDLC**).

Memory Trick

- Bit stuffing rule: after **five 1s**, stuff a **0**. Flag **01111110** = "**six 1s**".

CBT Trap

- Bit stuffing inserts **0** after **5** ones (not 6). Byte count method is the **least** robust. **Byte stuffing** is byte-oriented; **bit stuffing** is bit-oriented.

Error Detection and Correction

Error Types

- **Single-bit error**: only **one** bit flipped; rare in serial transmission.
- **Burst error**: **two or more** bits flipped; length = from first to last corrupted bit; common; favored by high data rate.

Hamming Distance

- **Hamming distance $d(x,y)$** = number of bit positions in which two codewords **differ** = **XOR then count 1s**.
- **Minimum Hamming distance $d_{\min}$** of a code determines power:
- To **detect** up to **s** errors: **$d_{\min} \geq s + 1$** .
- To **correct** up to **t** errors: **$d_{\min} \geq 2t + 1$** .

Memory Trick

- Detect $s \rightarrow$ need **$s+1$** ; correct $t \rightarrow$ need **$2t+1$** .

CBT Trap

- A code with **$d_{\min} = 3$** can **detect 2** and **correct 1**. Examiners ask "detect AND correct" $\rightarrow d_{\min} \geq s + t + 1$ with $s \geq t$.

Parity Check

- **Simple parity**: add **1** parity bit so total 1s are **even (even parity)** or **odd**; detects **odd** number of bit errors only (**cannot** detect even errors); **$d_{\min} = 2$** .
- **Two-dimensional parity**: arrange bits in a table, parity per **row and column**; detects more burst errors and can **correct single-bit** errors.

CBT Trap

- Simple parity **cannot** detect **2-bit (even)** errors and **cannot correct** anything. 2D parity can correct a **single** bit (intersection of failing row and column).

Checksum (Internet Checksum)

- Used at **transport/network** layers; sender sums data in **16-bit** words using **1's complement** arithmetic, takes **1's complement** of the sum as checksum.
- Receiver sums all words **including** checksum; result of **all 1s** (zero in 1's complement) means **no error**.
- Wraps around **end-around carry**; weaker than CRC; cannot detect some **reordering/multiple** errors.

CBT Trap

- Internet checksum uses **1's complement** sum, not 2's complement. A **carry-out** is added back (**wrap-around**). It cannot catch errors that **cancel out**.

Cyclic Redundancy Check (CRC)

- **CRC** uses **modulo-2 (XOR)** polynomial division; based on a **generator polynomial $G(x)$** of degree **r** .
- Procedure: append **r** zeros to data, divide by G (XOR), the **remainder** (r bits) is the **CRC/FCS**; transmit **data + CRC**.
- Receiver divides received frame by G ; remainder **0** = no detected error.
- Degree of generator **r** $\rightarrow$ CRC has **r** bits; codeword length = data + r .

CRC Detection Capability

- Detects **all single-bit** errors if G has **≥ 2 terms**.
- Detects all **burst errors** of length **$\leq r$** .
- Detects **odd** number of bit errors if G contains the factor **$(x + 1)$** .
- Burst of length **$r+1$** detected with prob **$1 - (1/2)^{(r-1)}$** ; longer bursts **$1 - (1/2)^r$** .

Memory Trick

- "Append r zeros, XOR divide, send remainder". CRC-32 (Ethernet) has $r = 32$.

CBT Trap

- CRC uses XOR (mod-2), no carries. Number of CRC bits = degree r of G , NOT number of terms. A burst $\leq r$ is always caught.

Hamming Code (Single-bit Correction)

- Hamming code is a SEC (single error correcting) block code; redundant parity bits placed at positions that are powers of 2: 1, 2, 4, 8, 16,
- Number of redundant bits r for m data bits satisfies $2^r \geq m + r + 1$.
- Parity bit at position 2^i checks all positions whose binary index has bit i set.
- Receiver recomputes parities $\rightarrow$ the binary syndrome points directly to the erroneous bit position (0 = no error).

Memory Trick

- Parity positions = powers of 2. Need r so that $2^r \geq m + r + 1$. For $m=7$ data bits $\rightarrow r=4 \rightarrow$ code (11,7).

CBT Trap

- For $m=7$: $2^4=16 \geq 7+4+1=12$ so $r=4$. Hamming corrects 1 bit, detects 2 (with extra overall parity = SECDED). Parity bits go at powers of 2, not the end.

Flow and Error Control Protocols

Stop-and-Wait

- **Stop-and-Wait**: sender sends **one** frame, waits for **ACK** before next; simplest flow control.
- **Efficiency (utilization)** = $1 / (1 + 2a)$ where $a = T_p / T_t$.
- **Throughput** = **Efficiency** x **Bandwidth**; sender window = receiver window = **1**.

Stop-and-Wait ARQ

- Adds **sequence numbers** (0/1, 1 bit), **timeouts**, and **retransmission** for lost/damaged frames; ACKs are numbered.
- Handles **lost frame**, **lost ACK** (retransmit on timeout), and **delayed ACK** (sequence numbers prevent duplicates).

Memory Trick

- $a = T_p / T_t$. Efficiency = $1/(1+2a)$. Sequence numbers need only **1** bit.

CBT Trap

- Stop-and-Wait window = **1**; efficiency $1/(1+2a)$. For large a (long links) efficiency is **poor** -> use sliding window.

Sliding Window: Go-Back-N ARQ

- **Go-Back-N (GBN)**: sender window $N = 2^m - 1$ (m = sequence-number bits); receiver window = **1**.
- Receiver accepts **in-order** only, sends **cumulative ACK**; on a lost frame the sender **retransmits the frame and all subsequent (go back N)**.
- **Efficiency** = $N / (1 + 2a)$, capped at **1** when $N \geq 1 + 2a$.

Sliding Window: Selective Repeat ARQ

- **Selective Repeat (SR)**: only the **specific** lost/damaged frame is retransmitted; receiver **buffers** out-of-order frames; uses **NAK**/selective ACK.
- Window constraint: **sender window = receiver window** $\leq 2^{(m-1)}$ (half the sequence space).
- **Efficiency** = $N / (1 + 2a)$ with the larger usable N .

Window Size vs Sequence-Number Bits

- With m sequence bits, sequence space = 2^m .
- **GBN**: max sender window = $2^m - 1$.
- **SR**: max window = $2^{(m-1)}$ (sender = receiver).

Feature	Go-Back-N	Selective Repeat
Sender window	$2^m - 1$	$2^{(m-1)}$
Receiver window	1	$2^{(m-1)}$
Retransmit	All after lost	Only lost
Receiver buffer	No	Yes
ACK type	Cumulative	Selective

Piggybacking

- **Piggybacking**: attach the **ACK** of incoming data onto an **outgoing data** frame to improve efficiency in **bidirectional** communication; adds slight **delay** waiting for data to send.

Efficiency / Throughput Computations

- $a = T_p / T_t$; $T_t = L/B$; $T_p = d/v$.
- **Utilization (sliding window)** = $N / (1 + 2a)$ (when < 1).
- **Throughput** = $U \times B$.

Optimal Window Size and BDP

- **Optimal N** = $1 + 2a$ so the pipe stays **full**; this equals $1 + BDP/L$.
- **BDP** = $B \times RTT = B \times 2T_p$ = max bits in flight.

Memory Trick

- "**1 + 2a**" is the magic number: efficiency $N/(1+2a)$, optimal window = **1+2a**.

CBT Trap

- **GBN** max window = $2^m - 1$; **SR** = $2^{(m-1)}$. If SR window $> 2^{(m-1)}$, sequence numbers **ambiguous**. For $m=3$: GBN=7, SR=4.

Medium Access Control (MAC)

Static vs Dynamic Allocation

- **Static**: fixed division (**FDM/TDM**); wasteful when traffic is **bursty**.
- **Dynamic**: allocate on demand; **random access**, **controlled access**, **channelization**.

Random Access: ALOHA

- **Pure ALOHA**: transmit **whenever** ready; vulnerable period = **2 x frame time**; max efficiency = **18.4% (1/2e)** at **G = 0.5**.
- **Slotted ALOHA**: transmit only at slot **start**; vulnerable period = **1 x frame time**; max efficiency = **36.8% (1/e)** at **G = 1**.
- Throughput **S** = $G \times e^{(-2G)}$ (pure), **S** = $G \times e^{(-G)}$ (slotted), where G = offered load.

Random Access: CSMA

- **CSMA** = **Carrier Sense Multiple Access**: "**listen before talk**".
- **1-persistent**: if idle, send **immediately** (prob 1); high collision.
- **Non-persistent**: if busy, wait a **random** time then sense again; fewer collisions, more delay.
- **p-persistent** (slotted): if idle, send with prob **p**, defer with **1-p**; balances both.

CSMA/CD

- **CSMA/CD** = collision **detection**; used in **wired Ethernet (802.3)**; "**listen while talking**", abort on collision, send **jam** signal, **binary exponential backoff**.
- **Minimum frame size** needed so a sender is still transmitting when collision returns: $T_t \geq 2 \times T_p$, i.e., $L \geq 2 \times B \times T_p$.
- After k collisions, backoff chosen from **0 to $2^k - 1$** (capped at **k=10**, max **16** attempts).

CSMA/CA

- **CSMA/CA** = collision **avoidance**; used in **wireless (802.11 Wi-Fi)** because collisions **cannot** be reliably detected (hidden terminal); uses **IFS**, **contention window/backoff**, and optional **RTS/CTS**; ACKs confirm.

Controlled Access

- **Reservation**: stations reserve slots in advance.
- **Polling**: a **primary** station invites secondaries to send (**poll/select**).
- **Token passing**: a **token** circulates; only the holder may transmit (**Token Ring, FDDI**).

Channelization

- **FDMA**: each station a **frequency band**.
- **TDMA**: each station a **time slot**.
- **CDMA**: all use **same frequency/time**, separated by orthogonal **codes (chip sequences)**.

Protocol	Type	Efficiency/Use
Pure ALOHA	Random	18.4%
Slotted ALOHA	Random	36.8%
CSMA/CD	Random	Wired Ethernet
CSMA/CA	Random	Wi-Fi
Token passing	Controlled	Deterministic

Memory Trick

- Slotted ALOHA **doubles** pure ALOHA: **18.4% -> 36.8%** ($1/2e \rightarrow 1/e$).

CBT Trap

- **CD** = wired Ethernet; **CA** = Wi-Fi. Min Ethernet frame **64 bytes** comes from **2 x Tp** requirement. Pure ALOHA vulnerable period is **2T**; slotted is **T**.

Ethernet and LAN Devices

IEEE 802.3 Ethernet Frame Format

- Fields: **Preamble (7 B)**, **SFD (1 B)**, **Destination MAC (6 B)**, **Source MAC (6 B)**, **Type/Length (2 B)**, **Data (46-1500 B)**, **FCS/CRC (4 B)**.
- Preamble (7 bytes)** = alternating 1010... for sync; **SFD = 10101011**.
- FCS = CRC-32**.

Min/Max Frame Size

- Minimum frame = **64 bytes** (512 bits) to support **CSMA/CD** collision detection.
- Maximum frame = **1518 bytes** (payload **1500 = MTU**); minimum payload **46 bytes** (padded if less).

MAC Addressing

- MAC address = 48 bits (6 bytes)**, shown as **12 hex digits**; first **24 bits = OUI** (vendor).
- Unicast**: LSB of first byte = **0**; **Multicast**: LSB = **1**; **Broadcast = FF:FF:FF:FF:FF:FF** (all 1s).
- MAC is **flat, burned-in (physical)**; does not change across the journey within a LAN.

Fast / Gigabit Ethernet

- Ethernet = 10 Mbps**; **Fast Ethernet = 100 Mbps**; **Gigabit = 1 Gbps**; **10G = 10 Gbps**.

Devices and Layers

- Hub/Repeater: Physical (L1)**; regenerates signal; **one collision domain, one broadcast domain**; no filtering.
- Bridge/Switch: Data Link (L2)**; **MAC learning** via source addresses, forwards by MAC table; each port = **separate collision domain**; switch = multiport bridge.
- Router: Network (L3)**; forwards by **IP**; separates **broadcast domains**; runs routing protocols.
- Gateway**: operates up to **Application (L7)**; **protocol converter** between dissimilar networks.

Device	Layer	Collision domains	Broadcast domains
Hub	L1	1	1
Switch	L2	per port	1 (per VLAN)
Router	L3	per port	per port

Collision vs Broadcast Domain

- Collision domain**: set of devices that can **collide**; separated by **switch/router/bridge**, NOT by hub.
- Broadcast domain**: reached by a **broadcast** frame; separated only by **router** (or VLAN).

Spanning Tree Protocol

- STP (802.1D)**: prevents **Layer-2 loops** by blocking redundant links to form a **loop-free tree**; elects a **root bridge** (lowest **Bridge ID**); uses **BPDUs**.

VLAN and 802.11 WLAN

- VLAN**: logically segments a switch into multiple **broadcast domains**; tag = **802.1Q (4 bytes, 12-bit VLAN ID)**.

- **IEEE 802.11** = **Wi-Fi WLAN**; uses **CSMA/CA**; modes **infrastructure (AP)** and **ad-hoc**.

Memory Trick

- "**Hub=1/1, Switch=many/1, Router=many/many**" (collision/broadcast domains).

CBT Trap

- Only a **router** (or **VLAN**) breaks a **broadcast domain**; switches break **collision** domains. **Switch** learns **source** MAC, forwards by **destination** MAC. Min Ethernet frame = **64 B**.

Network Layer Services

Core Functions

- **Logical addressing (IP)**, **routing** (path selection), and **forwarding** (moving a packet from input to output port).
- Provides **host-to-host** delivery; the Internet's network layer is **connectionless/best-effort (IP)**.

Forwarding vs Routing

- **Routing** = building the **routing table** (control plane, global, slow).
- **Forwarding** = the **per-packet** lookup and switch to output (data plane, local, fast).

CBT Trap

- **Routing** = make the table; **forwarding** = use the table. Control plane vs data plane.

Datagram vs Virtual Circuit (Network Layer)

- **Datagram**: each packet independent, **full destination IP**, may reorder; **IP**.
- **VC**: setup, **label-based**, ordered; **ATM/MPLS/Frame Relay**.

IPv4 Addressing

Address Structure

- **IPv4** = **32 bits** = **4 octets**, written **dotted decimal** (each octet **0-255**); total **2^{32} ~ 4.3 billion** addresses.
- Split into **network ID** + **host ID**.

Classful Addressing

Class	Leading bits	First octet	Default mask	Hosts
A	0	1-126	/8 (255.0.0.0)	$2^{24}-2$
B	10	128-191	/16 (255.255.0.0)	$2^{16}-2$
C	110	192-223	/24 (255.255.255.0)	2^8-2
D	1110	224-239	multicast	-
E	1111	240-255	reserved	-

- **127.x.x.x** = **loopback** (not class A usable); **0.x** reserved.

Network/Host IDs and Special Addresses

- **Network address**: host bits all **0**.
- **Broadcast address**: host bits all **1** (**directed broadcast**); **255.255.255.255** = limited broadcast.
- **Loopback**: **127.0.0.1**.
- Usable hosts = **$2^h - 2$** (subtract network + broadcast).

Private IP Ranges

- **10.0.0.0/8** (Class A), **172.16.0.0/12** (Class B), **192.168.0.0/16** (Class C); not routable on public Internet (use **NAT**).

- **169.254.0.0/16** = **APIPA** (link-local, auto).

Classless Addressing (CIDR)

- **CIDR** = **Classless Inter-Domain Routing**; uses **/n** prefix instead of fixed classes; mask = n leading **1s**.
- Block size = $2^{(32-n)}$ addresses; flexible allocation, reduces routing table size via **aggregation**.

Subnetting

- **Subnetting**: borrow **host** bits to create **subnets**; new mask longer.
- Number of subnets = 2^s (s = borrowed bits); hosts per subnet = $2^h - 2$.
- **Block size (increment)** = **256 - mask octet value**; subnet boundaries are multiples of the block size.
- **Subnet address** = host bits 0; **broadcast** = host bits 1; **valid range** = in between.

VLSM

- **VLSM** = **Variable-Length Subnet Masking**: subnets of **different** sizes from one block; allocate **largest first**; reduces address waste.

Supernetting / Route Aggregation

- **Supernetting**: combine contiguous networks into one **shorter-prefix** route; opposite of subnetting; used in **CIDR** to **aggregate** routes.

Address Block Problems

- Given an IP and mask: **network** = **IP AND mask**; **broadcast** = **network OR (NOT mask)**; **first host** = **network+1**; **last host** = **broadcast-1**.

Memory Trick

- **Block size** = **256 - mask**. Hosts = $2^h - 2$. Subnets = 2^s . **/30** gives **2** hosts (point-to-point).

CBT Trap

- Subtract **2** for usable hosts (network + broadcast). A **/31** (RFC 3021) allows **2** hosts with no broadcast (point-to-point). **/32** = single host. Classful default masks: A=**/8**, B=**/16**, C=**/24**.

IPv4 Datagram

Header Fields

- **IPv4 header** = 20 bytes minimum, 60 bytes max (with options); **Version (4 bits)**, **IHL**, **Type of Service/DSCP**, **Total Length (16 bits)**, **Identification**, **Flags (3 bits)**, **Fragment Offset (13 bits)**, **TTL (8 bits)**, **Protocol (8 bits)**, **Header Checksum**, **Source IP**, **Destination IP**, **Options**.
- **IHL** counts header length in **4-byte words** (min 5 = 20 B, max 15 = 60 B).
- **Total Length** = header + data, max 65535 bytes.

TTL, Protocol, Checksum, Total Length

- **TTL**: decremented by each **router**; at 0 packet dropped, ICMP **Time Exceeded** sent; prevents **loops**.
- **Protocol** field: 1 = ICMP, 6 = TCP, 17 = UDP, 2 = IGMP.
- **Header checksum**: covers **header only**; recomputed at **each hop** (because TTL changes).

Fragmentation and Reassembly

- **MTU** = max frame payload of a link (**Ethernet = 1500 B**); larger packets are **fragmented**.
- **Identification**: same for all fragments of a packet.
- **Flags**: **DF (Don't Fragment)**, **MF (More Fragments)** = 1 on all but last.
- **Fragment Offset**: position of fragment in original, in units of **8 bytes**.
- Each fragment's data (except last) must be a **multiple of 8 bytes**.
- Reassembly done only at **destination**.

Memory Trick

- Offset is in **8-byte** units. Protocol: **TCP=6, UDP=17, ICMP=1**. Header min/max = **20/60**.

CBT Trap

- **Fragment offset** counts **8-byte** units, so each non-last fragment payload is a multiple of 8. **MF=0** on the **last** fragment. **TTL** change forces checksum **recompute** at every router. Reassembly only at the **destination**, never intermediate.

IPv6

128-bit Addressing

- **IPv6** = 128 bits = 8 groups of 4 hex digits, colon-separated; 2^{128} addresses.
- Shorthand: drop **leading zeros**; one run of zeros -> **::** (only once).

Header Simplification

- **IPv6 header** = fixed 40 bytes; fewer fields; **no checksum**, **no fragmentation by routers** (only source), no header length field; uses **extension headers**.
- **TTL** renamed **Hop Limit**; **Protocol** -> **Next Header**; adds **Flow Label**.

Transition

- **Dual stack**: run **IPv4 and IPv6** together.
- **Tunneling**: encapsulate **IPv6 in IPv4**.
- **Translation (NAT64)**: convert between versions.

CBT Trap

- IPv6 has **no header checksum** and routers do **not fragment**. Header is **fixed 40 B** (vs IPv4 variable **20-60**). **::** can appear only **once**.

Routing Algorithms

Static vs Dynamic Routing; Metrics

- **Static routing**: manually configured; no overhead; fails on **topology change**; good for small/stub networks.
- **Dynamic routing**: routers **exchange** info, adapt automatically; uses **metrics** = **hop count, bandwidth, delay, cost, load, reliability**.

Shortest Path Routing (Dijkstra)

- **Dijkstra's algorithm**: computes **single-source shortest paths** on non-negative weights; greedy, picks nearest unvisited node.
- Complexity: $O(V^2)$ with array, $O((V+E) \log V)$ with binary heap; the basis of **Link State** routing.

Flooding

- **Flooding**: every incoming packet sent on **all** links except the one it arrived on; guarantees delivery but **duplicates**; controlled by **TTL/hop count** or **sequence numbers**.

Distance Vector Routing

- **Distance Vector (DV)**: each router shares its **whole table** (vector of distances) with **neighbors** only (**Bellman-Ford** basis); "**routing by rumor**".
- **Bellman-Ford**: $D_x(y) = \min \text{ over neighbors } v \text{ of } [c(x,v) + D_v(y)]$.
- **Count-to-infinity**: bad news propagates slowly; loops cause metrics to climb to **infinity (16 in RIP)**.
- **Split horizon**: don't advertise a route back on the interface you **learned** it; **poison reverse**: advertise it with **infinite** metric.
- **RIP**: DV protocol, metric = **hop count**, max **15 (16 = infinity/unreachable)**, updates every **30 s**, uses **UDP port 520**.

Link State Routing

- **Link State (LS)**: each router **floods** its link costs (**LSP**) to **all** routers, builds full **topology database**, runs **Dijkstra** locally.
- Converges fast, no count-to-infinity; more **memory/CPU**.
- **OSPF**: LS protocol, metric = **cost (bandwidth-based)**, uses **areas**, IP protocol number **89**, supports **VLSM**.

Hierarchical Routing, AS, BGP

- **Autonomous System (AS)**: a network under single administrative control, has an **AS number**.
- **IGP** (within AS): **RIP, OSPF**; **EGP** (between AS): **BGP**.
- **BGP**: **path-vector** protocol, the routing protocol of the **Internet**, uses **TCP port 179**.

Feature	Distance Vector	Link State
Shares	Whole table	Link states (LSP)
With	Neighbors	All routers
Algorithm	Bellman-Ford	Dijkstra
Convergence	Slow	Fast
Example	RIP	OSPF

Feature	Distance Vector	Link State
Problem	Count-to-infinity	High memory

Memory Trick

- DV = "tell neighbors everything"; LS = "tell everyone your links". RIP hop max = **15**, infinity = **16**.

CBT Trap

- RIP**=DV/hop count/port **520**; **OSPF**=LS/cost/proto **89**; **BGP**=path-vector/**TCP 179**. **Count-to-infinity** afflicts DV, not LS. Dijkstra needs **non-negative** weights.

IP Support Protocols

ARP (Address Resolution Protocol)

- ARP**: maps a known **IP** to a **MAC** address within a LAN; **broadcast** request ("who has IP?"), **unicast** reply.
- Results cached in **ARP cache** (timeout-based); operates between **L2 and L3**.
- Gratuitous ARP**: announce own mapping (detect duplicate IP).

RARP

- RARP**: maps **MAC -> IP** (reverse); obsolete, replaced by **BOOTP/DHCP**.

ICMP

- ICMP**: **error reporting** and diagnostics for IP; encapsulated in **IP** (protocol **1**); not for data.
- Messages: **Echo request/reply (ping)**, **Destination Unreachable**, **Time Exceeded (TTL=0)**, **Redirect**, **Source Quench**.
- ping**: tests reachability (echo); **traceroute/tracert**: increments **TTL** to discover each hop via **Time Exceeded**.

DHCP

- DHCP**: dynamically assigns **IP, mask, gateway, DNS**; client **UDP port 68**, server **UDP port 67**.
- DORA**: **Discover -> Offer -> Request -> Acknowledge**.
- Lease**: time-limited assignment; renewed at **T1 (50%)** and **T2 (87.5%)**.

NAT

- NAT**: translates **private** addresses to **public**; conserves IPv4.
- Static NAT**: 1-to-1 fixed; **Dynamic NAT**: from a pool; **PAT/overloading**: many-to-one using **port numbers** (most common, home routers).

Memory Trick

- DORA** = Discover, Offer, Request, Ack. DHCP ports = **67 server / 68 client**. ARP = **IP->MAC**; RARP = **MAC->IP**.

CBT Trap

- ARP** resolves **IP->MAC** (forward); **RARP** the reverse. **ICMP** reports errors but does NOT correct them and is **not** a transport protocol. DHCP server=**67**, client=**68** (often swapped). **traceroute** exploits **TTL** expiry.

Transport Layer Services

Process-to-Process Delivery; Ports and Sockets

- Transport layer gives **process-to-process** (end-to-end) delivery; network layer only does **host-to-host**.
- **Port number** = **16 bits (0-65535)** identifies a process; **socket** = **IP + port**.
- A connection is identified by the **4-tuple**: **source IP, source port, dest IP, dest port**.

Multiplexing and Demultiplexing

- **Multiplexing**: gather data from many sockets, add headers, pass down.
- **Demultiplexing**: deliver incoming segments to the correct **socket** using port numbers (and 4-tuple for TCP).

Connection-oriented vs Connectionless

- **TCP** = connection-oriented, reliable, ordered; **UDP** = connectionless, unreliable, fast.

UDP

UDP Header

- **UDP header** = fixed **8 bytes**: **Source Port (2)**, **Destination Port (2)**, **Length (2)**, **Checksum (2)**.
- **Length** = header + data; minimum **8**.

UDP Checksum

- **Optional** in IPv4 (0 = unused), **mandatory** in IPv6; computed over **pseudo-header + UDP header + data**.

Use Cases

- **DNS, DHCP, TFTP, streaming, VoIP, online gaming, SNMP** - where **speed** beats reliability.

CBT Trap

- UDP header is always **8 bytes** (4 fields). UDP has **no** sequencing, flow, or congestion control. Checksum is **optional** in IPv4 only.

TCP

TCP Header Fields

- **TCP header** = **20 bytes** min, **60** max; **Source/Dest Port (2 each)**, **Sequence Number (4)**, **Acknowledgment Number (4)**, **Data Offset/HLEN (4 bits)**, **Flags**, **Window Size (2)**, **Checksum (2)**, **Urgent Pointer (2)**, **Options**.
- Flags: **URG, ACK, PSH, RST, SYN, FIN** (and ECE/CWR).

Sequence and Acknowledgment Numbers

- TCP is **byte-oriented**: sequence number = byte number of **first byte** in segment.

- ACK number = **next expected byte** (**cumulative**).
- Initial sequence number (**ISN**) is **random**.

Connection Management

- **Three-way handshake**: **SYN -> SYN+ACK -> ACK**; exchanges **ISNs** and options (**MSS, window scale**).
- **Connection termination**: **four-way** (**FIN -> ACK -> FIN -> ACK**); each direction closed independently (**half-close**).
- **TCP states**: **LISTEN, SYN-SENT, SYN-RECEIVED, ESTABLISHED, FIN-WAIT, TIME-WAIT (2*MSL), CLOSED**.

Flow Control

- TCP uses a **sliding window**; receiver advertises **rwnd (advertised window)**; sender limited by **min(rwnd, cwnd)**.
- Prevents **receiver** buffer overflow; **zero window** pauses sender; **persist timer** probes.

Error Control

- **Retransmission** on timeout (**RTO**) or **3 duplicate ACKs**; **cumulative ACK**; checksum detects corruption.
- **RTO** computed from **RTT** via **Jacobson/Karn (EstimatedRTT, DevRTT)**.

Congestion Control

- Goal: avoid overwhelming the **network** (vs flow control = receiver).
- **Congestion window (cwnd)**: sender's estimate of safe in-flight data.
- **Slow start**: cwnd starts at **1 MSS**, **doubles** each RTT (**exponential**) until **ssthresh**.
- **Congestion avoidance (AIMD)**: above ssthresh, cwnd grows **linearly** (+1 MSS/RTT) - **additive increase**; on loss, **multiplicative decrease**.
- **Timeout**: ssthresh = **cwnd/2**, cwnd = **1** (back to slow start).
- **Fast retransmit**: **3 duplicate ACKs** -> retransmit immediately without waiting for timeout.
- **Fast recovery**: after 3 dup ACKs, ssthresh = cwnd/2, cwnd = ssthresh (skip slow start) - **TCP Reno**.

cwnd Evolution Computation

- Slow start: **1, 2, 4, 8, 16...** (double per RTT) until **ssthresh**.
- After ssthresh: **+1 per RTT** linear.
- On **timeout**: ssthresh = half of current cwnd, cwnd = **1**.
- On **3 dup ACK**: ssthresh = half, cwnd = ssthresh (Reno).

TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Reliability	Reliable	Unreliable
Ordering	Yes	No
Header	20-60 B	8 B
Speed	Slower	Faster
Flow/Congestion	Yes	No
Use	HTTP, FTP, SMTP	DNS, VoIP, streaming

Memory Trick

- Slow start = **exponential**; congestion avoidance = **linear (AIMD)**. **Timeout** -> cwnd=1; **3 dup ACK** -> fast recovery. Handshake = **3-way**, teardown = **4-way**.

CBT Trap

- **Flow control** protects **receiver**; **congestion control** protects **network**. Slow start is **exponential** (misleading name). On **timeout** cwnd resets to **1**; on **3 dup ACKs** (Reno) it halves. TCP header **20-60 B**, UDP fixed **8 B**. TIME-WAIT = **2*MSL**.

Sockets

Socket = IP + Port

- A **socket** uniquely identifies a network endpoint as **IP address + port number**.

Port Ranges

- **Well-known ports: 0-1023** (assigned by IANA: HTTP 80, etc.).
- **Registered ports: 1024-49151**.
- **Dynamic/ephemeral/private ports: 49152-65535** (client side).

CBT Trap

- Well-known **0-1023**; registered **1024-49151**; ephemeral **49152-65535**. Port field is **16 bits** (max **65535**).

DNS (Domain Name System)

Namespace Hierarchy

- **DNS** maps **domain names** to **IP addresses**; a **distributed, hierarchical** database.
- Hierarchy: **root (.)** -> **TLD (.com, .org, .in)** -> **authoritative (example.com)** servers.
- **13** logical **root server** clusters worldwide.

Record Types

Record	Maps
A	name -> IPv4
AAAA	name -> IPv6
MX	mail exchange server
CNAME	alias -> canonical name
NS	name server
PTR	IP -> name (reverse)

Iterative vs Recursive Resolution

- **Recursive**: resolver does all the work and returns the **final** answer to the client.
- **Iterative**: each server returns a **referral** to the next server; the client/resolver chases it.

DNS Caching, TTL, Port

- Answers cached for the record's **TTL**; reduces load and latency.
- DNS uses **UDP port 53** for queries, **TCP port 53** for **zone transfers** and large responses.

Memory Trick

- **A=IPv4, AAAA=IPv6, MX=mail, CNAME=alias, NS=nameserver, PTR=reverse**. DNS port = **53**.

CBT Trap

- DNS uses **UDP 53** normally but **TCP 53** for zone transfers / responses > 512 B. **CNAME** is an alias, **PTR** is reverse lookup. Recursive vs iterative is who does the **chasing**.

Email Protocols

Email Architecture

- **MUA (Mail User Agent)**: client (Outlook); **MTA (Mail Transfer Agent)**: server relay (SMTP); **MDA (Mail Delivery Agent)**: delivers to mailbox.

SMTP

- **SMTP**: **push** protocol, sends mail between servers and from client to server; **TCP port 25** (or **587** submission, **465** SMTPS).

POP3

- **POP3**: **pull**; downloads and usually **deletes** mail from server; **TCP port 110** (POP3S **995**); no sync across devices.

IMAP

- **IMAP**: **pull**; keeps mail on server, supports **folders/sync** across devices; **TCP port 143** (IMAPS **993**).

MIME

- **MIME**: extends email to carry **non-ASCII** data (**images, attachments**); adds headers like **Content-Type**.

Protocol	Port	Role
SMTP	25	Push/send
POP3	110	Pull/download
IMAP	143	Pull/sync

CBT Trap

- **SMTP=push (25)**, **POP3=110**, **IMAP=143**. POP3 **deletes** from server; IMAP **syncs**. MIME enables **attachments**, it is not a transfer protocol.

Web (HTTP/HTTPS)

Request/Response Model

- **HTTP**: client-server, **request/response**, runs over **TCP port 80** (**HTTPS port 443**); **stateless**.

Methods

- **GET** (retrieve), **POST** (submit), **PUT** (replace/create), **DELETE**, **HEAD** (headers only), **PATCH**, **OPTIONS**.
- **GET** is **idempotent/safe**; **POST** is **not** idempotent.

Status Codes

Class	Meaning	Example
1xx	Informational	100 Continue
2xx	Success	200 OK
3xx	Redirection	301/304
4xx	Client error	404, 403, 400
5xx	Server error	500, 502, 503

Persistent vs Non-persistent

- **Non-persistent (HTTP/1.0)**: one **TCP connection per object**; **2 RTT** + transmit per object.
- **Persistent (HTTP/1.1)**: reuse one connection (**keep-alive**), supports **pipelining**; default.

Stateless; Cookies/Sessions

- HTTP is **stateless**; state added by **cookies** (client-side token) and **sessions** (server-side).

HTTPS/TLS

- **HTTPS** = HTTP over **TLS/SSL**; encrypts traffic; **port 443**; provides **confidentiality, integrity, authentication**.

Memory Trick

- HTTP=**80**, HTTPS=**443**. **2xx OK**, **3xx redirect**, **4xx you erred**, **5xx server erred**. **404**=not found, **500**=server error.

CBT Trap

- **GET** idempotent, **POST** not. **301**=permanent redirect, **302**=temporary, **304**=not modified. HTTP **1.1** is persistent by default. HTTP is **stateless** -> cookies add state.

File Transfer (FTP)

Control vs Data Connection

- **FTP** uses **two** TCP connections: **control = port 21** (commands), **data = port 20** (file transfer); control is **persistent**, data is **per-transfer**.

Active vs Passive Mode

- **Active**: server initiates the **data** connection back to client (from port **20**); problematic with client **firewalls/NAT**.
- **Passive**: client initiates **both** connections; firewall-friendly.

ASCII vs Binary Transfer

- **ASCII mode**: text, converts line endings; **Binary/image mode**: raw bytes for non-text files.

CBT Trap

- FTP control = **21**, data = **20**. **Active** = server connects back; **Passive** = client connects (NAT-friendly). FTP keeps **control** open through the session (**out-of-band** control).

Other Services

Telnet / SSH

- **Telnet**: remote login, **plaintext**, **TCP port 23** - insecure.
- **SSH**: secure encrypted remote login/tunneling, **TCP port 22** - replaces Telnet.

SNMP

- **SNMP**: **Simple Network Management Protocol**; manager/agent, **MIB**, uses **UDP ports 161 (agent) / 162 (trap)**.

Proxy / Web Caching

- **Proxy/cache**: intermediary that **caches** responses, reduces latency/bandwidth, can filter; **forward proxy** (client side) vs **reverse proxy** (server side).

CBT Trap

- **Telnet 23** (insecure) vs **SSH 22** (secure). **SNMP = UDP 161/162**. Proxy cache reduces **bandwidth** and **latency**.

Network Security: Foundations

Security Goals (CIA + AN)

- **Confidentiality**: only authorized parties can **read** data (**encryption**).
- **Integrity**: data not **altered** undetected (**hashing/MAC**).
- **Availability**: services remain **accessible** (anti-DoS).
- **Authentication**: verify **identity** of a party.
- **Non-repudiation**: sender **cannot deny** having sent (digital signatures).
- **Authorization/Access control**: what an authenticated party may **do**.

Memory Trick

- **CIA** = **Confidentiality, Integrity, Availability**. Add **Authentication** + **Non-repudiation** for the full set.

CBT Trap

- **Encryption** provides **confidentiality**, NOT integrity by itself. **Hashing** provides **integrity**, not confidentiality. **Digital signature** provides **authentication + integrity + non-repudiation**.

Active vs Passive Attacks

- **Passive**: eavesdropping, **traffic analysis** - hard to detect, easy to prevent (encryption).
- **Active**: **modification, masquerade/spoofing, replay, DoS** - easy to detect, hard to prevent.

Cryptography

Symmetric-Key Cryptography

- **Symmetric (secret-key)**: **same key** for encryption and decryption; **fast**; problem = **key distribution**.
- For **n** users, symmetric needs **$n(n-1)/2$** keys.
- Types: **block ciphers** (DES, AES) and **stream ciphers** (RC4).
- Modes: **ECB** (insecure, patterns leak), **CBC** (chaining + IV), **CTR**, **CFB/OFB**.

DES (Data Encryption Standard)

- **DES**: block size **64 bits**, key **56 bits** (64 with **8 parity** bits), **16 Feistel rounds**.
- Now **insecure** (key too short, brute-forceable).
- **3DES (Triple DES)**: applies DES **3 times**, effective key **112/168 bits**.

AES (Advanced Encryption Standard)

- **AES**: block size **128 bits**; key sizes **128/192/256 bits**; rounds **10/12/14** respectively.
- Based on **substitution-permutation network** (not Feistel); operations: **SubBytes, ShiftRows, MixColumns, AddRoundKey**.
- Successor to DES (**Rijndael**); current standard.

Cipher	Block	Key	Rounds	Structure
DES	64	56	16	Feistel
3DES	64	112/168	48	Feistel x3

Cipher	Block	Key	Rounds	Structure
AES	128	128/192/256	10/12/14	SPN

Asymmetric-Key (Public-Key) Cryptography

- **Asymmetric: key pair** - **public key** (shared) and **private key** (secret); **slow**; solves **key distribution**.
- For **n** users needs only **2n** keys.
- **Confidentiality**: encrypt with **receiver's public** key, decrypt with **receiver's private**.
- **Authentication/signature**: encrypt (sign) with **sender's private** key, verify with **sender's public**.
- Examples: **RSA**, **Diffie-Hellman**, **ECC**, **ElGamal**.

RSA Algorithm

- **RSA**: based on difficulty of **factoring large primes**.
- Steps: pick primes **p, q**; **n = p x q**; **phi(n) = (p-1)(q-1)**; choose **e** with **gcd(e, phi)=1**; compute **d = e⁻¹ mod phi(n)**.
- Public key = **(e, n)**; private key = **(d, n)**.
- Encrypt: **C = M^e mod n**; Decrypt: **M = C^d mod n**.

Diffie-Hellman Key Exchange

- **Diffie-Hellman**: securely establishes a **shared secret** over an insecure channel; based on **discrete logarithm** problem.
- Vulnerable to **man-in-the-middle** without authentication.

Feature	Symmetric	Asymmetric
Keys	Same (1)	Pair (public/private)
Speed	Fast	Slow
Keys for n users	$n(n-1)/2$	2n
Key distribution	Hard	Easy
Examples	DES, AES, RC4	RSA, DH, ECC

Memory Trick

- **Confidentiality** -> encrypt with **receiver public**. **Signature** -> sign with **sender private**. Symmetric keys = $n(n-1)/2$; asymmetric = **2n**.

CBT Trap

- For **confidentiality**: use **receiver's public** key. For **authentication**: use **sender's private** key. RSA security = **factoring**; DH = **discrete log**. AES block always **128 bits** even with 256-bit key. DES key = **56** effective bits.

Hashing and Integrity

Cryptographic Hash Functions

- **Hash function**: maps arbitrary input to **fixed-size** digest; **one-way** (irreversible) and **collision-resistant**.
- Properties: **pre-image resistance**, **second pre-image resistance**, **collision resistance**, **avalanche effect**.
- **MD5**: **128-bit** digest - **broken** (collisions).
- **SHA-1**: **160-bit** - **deprecated**.
- **SHA-256** (SHA-2 family): **256-bit** - current.

MAC and HMAC

- **MAC (Message Authentication Code)**: hash with a **shared secret key** -> provides **integrity + authentication** (not non-repudiation).
- **HMAC**: keyed-hash MAC construction.

CBT Trap

- **Hash** alone gives **integrity** only; **MAC** adds **authentication** but **not** non-repudiation (shared key). **MD5=128 bit**, **SHA-1=160**, **SHA-256=256**. Hash is **one-way**, not encryption.

Digital Signatures and Certificates

Digital Signatures

- **Digital signature**: sender **hashes** the message, then **encrypts the hash with their private key**; receiver decrypts with sender's **public key** and compares hashes.
- Provides **authentication, integrity, non-repudiation**; does NOT provide **confidentiality** (unless combined with encryption).
- Algorithms: **RSA, DSA, ECDSA**.

Digital Certificates and PKI

- **Certificate**: binds a **public key** to an **identity**, signed by a **CA (Certificate Authority)**; standard **X.509**.
- **PKI**: hierarchy of **CAs, registration authorities, CRL** (revocation), trust anchors (**root CA**).

Memory Trick

- Sign = "**hash then encrypt with private key**". Verify = "**decrypt with public key, compare hash**". Certificate = **public key + identity** signed by **CA**.

CBT Trap

- Digital signature uses **sender's private key** on the **hash**, not the whole message (for speed). It does **not** give confidentiality. **X.509** = certificate standard. **CA** vouches for the public key.

SSL/TLS

SSL/TLS Overview

- **SSL/TLS**: secures application traffic (**HTTPS**); sits between **Application** and **Transport**; **TLS** succeeds **SSL**.
- Provides **confidentiality (symmetric)**, **authentication (certificates/asymmetric)**, **integrity (MAC)**.
- **TLS handshake**: negotiate cipher, authenticate server (certificate), exchange keys (**asymmetric**) to derive a **symmetric session key**, then bulk encrypt with **symmetric**.
- Sub-protocols: **Handshake, Record, Alert, Change Cipher Spec**.

CBT Trap

- TLS uses **asymmetric** only for the **handshake/key exchange**, then **symmetric** for bulk data (speed). HTTPS = **port 443**. TLS provides all of **C, I, A**.

Firewalls and IDS

Firewalls

- **Firewall**: filters traffic between networks by **rules/policy**; placed at the **perimeter**.
- **Packet-filtering** firewall: checks **IP/port/protocol** headers (stateless, L3/L4).
- **Stateful inspection**: tracks **connection state**.
- **Application/proxy** firewall: inspects **application-layer** content (L7).
- **DMZ**: a buffer subnet for public-facing servers.

IDS / IPS

- **IDS (Intrusion Detection System)**: **detects** and alerts (passive).
- **IPS (Intrusion Prevention System)**: detects and **blocks** (active, inline).
- Detection methods: **signature-based** (known patterns) and **anomaly-based** (deviation from baseline).

Type	Layer	Inspects
Packet filter	L3/L4	Headers
Stateful	L3/L4	Connection state
Proxy/App	L7	Content

CBT Trap

- **IDS** = detect/alert (passive); **IPS** = block (inline). **Signature-based** catches **known** attacks; **anomaly-based** catches **unknown/zero-day** but more **false positives**. Packet filter = **stateless L3/L4**.

Common Attacks

Attack Catalog

- **DoS/DDoS**: flood to exhaust resources (**availability**).
- **Man-in-the-Middle (MITM)**: intercept/alter between parties.
- **Replay**: resend captured valid messages (countered by **nonce/timestamp**).
- **Spoofing**: forge **IP/MAC/email** identity.
- **ARP poisoning**: fake ARP replies to redirect LAN traffic.
- **DNS spoofing/cache poisoning**: inject false DNS records.
- **Phishing**: trick users into revealing credentials.
- **SQL injection**: inject SQL via input.
- **XSS**: inject scripts into web pages.
- **Brute force / dictionary**: guess keys/passwords.
- **Smurf**: ICMP broadcast amplification; **SYN flood**: half-open TCP connections.

Attack	Targets	Countermeasure
DDoS	Availability	Rate limit, filtering
MITM	Confidentiality	TLS , auth
Replay	Integrity	Nonce/timestamp
ARP poison	LAN	Static ARP, DAI
SYN flood	TCP	SYN cookies

Memory Trick

- **Replay** -> defend with **nonce/timestamp**. **SYN flood** -> **SYN cookies**. **MITM** -> **TLS + certificates**.

CBT Trap

- **DoS** attacks **availability**; **MITM** attacks **confidentiality/integrity**. **SYN flood** exploits the **3-way handshake** (half-open). **ARP poisoning** works only on the **local LAN**. **Replay** defeated by **nonce**.

CIL MT Exam Focus / Practice Checklist

Port Numbers and Protocol-to-Layer Mapping

Port	Protocol	Transport	Layer
20/21	FTP (data/control)	TCP	App
22	SSH	TCP	App
23	Telnet	TCP	App
25	SMTP	TCP	App
53	DNS	UDP/TCP	App
67/68	DHCP (server/client)	UDP	App
69	TFTP	UDP	App
80	HTTP	TCP	App
110	POP3	TCP	App
143	IMAP	TCP	App
161/162	SNMP	UDP	App
179	BGP	TCP	App
443	HTTPS	TCP	App
520	RIP	UDP	App

- Protocol numbers (IP header): **ICMP=1**, **IGMP=2**, **TCP=6**, **UDP=17**, **OSPF=89**.

Formula Quick Sheet

- Transmission delay** = L/B ; **Propagation delay** = d/v ; **a** = T_p/T_t .
- Stop-and-Wait efficiency** = $1/(1+2a)$.
- Sliding window efficiency** = $N/(1+2a)$; **optimal N** = $1+2a$.
- BDP** = Bandwidth x RTT.
- Nyquist**: $C = 2B \log_2 L$; **Shannon**: $C = B \log_2(1+S/N)$.
- Bit rate** = baud x $\log_2 L$.
- Hosts per subnet** = $2^h - 2$; **subnets** = 2^s ; **block size** = $256 - \text{mask octet}$.
- GBN window** = $2^m - 1$; **SR window** = $2^{(m-1)}$.
- Mesh links** = $n(n-1)/2$.
- Pure ALOHA** = 18.4%; **Slotted ALOHA** = 36.8%.
- Symmetric keys** = $n(n-1)/2$; **Asymmetric keys** = $2n$.

Subnetting / CIDR Computations

- Steps: identify **mask** -> **block size** = $256 - \text{mask}$ -> find subnet the IP falls in -> **network/broadcast/range**.
- /30** = 4 addresses, 2 usable (point-to-point links).
- /24** = 256 addresses, 254 hosts.

Fragmentation Offset/Size Computations

- Offset** in 8-byte units; each non-last fragment data multiple of 8.
- Number of fragments for data D and per-fragment payload P = **ceil(D/P)**.

Sliding Window Efficiency

- Compute $a = Tp/Tt$, then $N/(1+2a)$; need at least m bits where $2^m \geq N+1$ (GBN).

TCP Congestion Window Evolution

- Track **cwnd** per RTT: **slow start doubles** until ssthresh, then **+1 linear**; reset on loss.

CRC and Hamming Computation

- CRC: append r zeros, **XOR divide**, remainder = FCS.
- Hamming: $2^r \geq m+r+1$, parity at **powers of 2**, syndrome locates error.

Delay / Throughput / BDP

- Total delay = sum of **transmission + propagation + queuing + processing** over all hops.
- **Throughput = efficiency x bandwidth**.

Header Field Identification

- **IPv4 = 20-60 B**, **TCP = 20-60 B**, **UDP = 8 B**, **IPv6 = 40 B fixed**, **Ethernet header = 14 B + 4 B FCS**.

Memory Trick

- "**20-20-8-40**" = IPv4, TCP, UDP, IPv6 header sizes. Ports: "**21 22 23 25 53 80 110 143 443**".

CBT Trap

- **DNS** and **DHCP** use **UDP** (DNS also TCP for zone transfers). **IPv6** header **fixed 40 B** with **no checksum**. Usable hosts subtract **2**. Offset in **8-byte** units. **Slow start** is exponential.